DJANGO TEXTBOOK SERIES [FILE 01] · COMPLETE CORE SUMMARY & CODE BIBLE (LECTURES 1-11)

[1번째 파일] 한눈에 보는 Django 전체 핵심 요점정리

교재 1권~10권의 필수 터미널 명령어 · 표준 코드 공식 · 파일별 작성 순서 · 베테랑 개발자의 실전 디버깅 체크리스트를 단원별로 총망라한 실전 요약집

본 문서는 Django 3종 학습 시리즈 중 **1번째 파일(전체 핵심 요점정리)**입니다. 3번째 파일(그림으로 이해시키는 교재)에서 '왜 이렇게 작동하는가'라는 원리를 그림으로 깨쳤다면, 이 요점정리 파일은 실제 프로젝트를 구현하거나 시험을 준비할 때 **"어느 파일에 어떤 코드를 어떤 순서로 작성해야 하는가"**를 한눈에 확인하고 즉시 적용할 수 있도록 시니어 개발자 관점에서 압축 정리한 실전 바이블입니다.

단원 구분	핵심 주제 (교재 1~11권)	주요 수정·작성 대상 파일	핵심 산출물 및 필수 마스터 목표
제 1 단원 (1권)	가상환경 · 프로젝트 초기화 · MTV	`settings.py`, `urls.py`, `views.py`	독립 가상환경 구축 및 첫 화면 렌더링 (`render`)
제 2 단원 (2권)	DTL 문법 · 템플릿 상속 · URL 분리	`base.html`, `app/urls.py`, `views.py`	공통 레이아웃 상속(`extends`) 및 `include`·역참조
제 3 단원 (3권)	Model 정의 · 마이그레이션 · Admin	`models.py`, `migrations/`, `admin.py`	DB 테이블 설계도 생성·시공 및 관리자 페이지 등록
제 4 단원 (4권)	UI/UX 법칙 · 웹 폰트 · 반응형 웹	`base.html`(`viewport`), `style.css`(`@media`)	모바일·태블릿·PC 반응형 레이아웃 및 UX 최적화
제 5 단원 (5권)	ORM · QuerySet API · Field Lookup	`shell_plus`, `views.py`	SQL 없이 파이썬 객체로 DB CRUD 및 조건 검색
제 6 단원 (6권)	ORM with View (수동 7대 CRUD)	`views.py`, `templates/`(`csrf_token`)	`GET` 조회와 `POST` 생성·수정·삭제 + `redirect`
제 7 단원 (7권)	Django Form · ModelForm · View 통합	`forms.py`, `views.py`(`create`, `update`)	서버 유효성 검사(`is_valid`) 및 단일 View 분기
제 8·9 단원 (8·9권)	쿠키·세션 · Custom User · 로그인/가입/탈퇴	`accounts/models.py`, `forms.py`, `views.py`	`AbstractUser` 교체, 로그인·로그아웃·회원가입·탈퇴
제 10 단원 (10권)	회원수정 · 비번변경 · 해시암호화 · 비번초기화	`accounts/forms.py`, `views.py`, `crud/urls.py`	`CustomUserChangeForm`, `PasswordChangeForm`, `PBKDF2`, `auth.urls`
제 11 단원 (11권)	Model·Form·Auth 실전 심화 (PJT-04)	`todos/models.py`, `forms.py`, `accounts/views.py`	`choices`, `cleaned_data`, `MultipleChoiceField`, `messages`, `?next=` 보안

제 1 단원 : 프로젝트 초기 세팅과 MTV 기본 흐름 요점정리 (교재 1권: Intro & Design Pattern)

1. 프로젝트 시작 시 무손실 6단계 터미널 명령어 루틴 (순서 엄수!)

순서	터미널 입력 명령어	핵심 역할 및 베테랑 개발자의 실전 주의사항
1단계	`python -m venv venv`	현재 폴더에 `venv`라는 이름의 독립 가상환경 폴더 생성 (`.gitignore`에 `venv/` 등록 필수)
2단계	• Git Bash: `source venv/Scripts/activate` • CMD: `venv\Scripts\activate` • Mac/Linux: `source venv/bin/activate`	가상환경 활성화. 반드시 터미널 프롬프트에 `(venv)`가 떴는지 확인한 후 다음 단계 진행! (비활성화 명령어는 `deactivate`)

순서	터미널 입력 명령어	핵심 역할 및 베테랑 개발자의 실전 주의사항
3단계	`pip install django` `pip freeze > requirements.txt`	Django 설치 후 현재 패키지 목록(레시피)을 `requirements.txt`로 추출 (팀원이 내려받았을 땐 `pip install -r requirements.txt` 실행)
4단계	`django-admin startproject firstpjt .`	**[주의]** 맨 뒤의 마침표(`.`) 필수! 점을 찍어야 현재 폴더 바로 아래에 `manage.py`가 생기며, 점을 빼먹으면 `firstpjt/firstpjt/`처럼 폴더가 이중으로 중첩됨
5단계	`python manage.py startapp articles`	기능 단위 모듈인 `articles` 앱 생성. **반드시 앱을 먼저 생성한 뒤 `settings.py`에 등록**해야 함!
6단계	`python manage.py runserver`	개발용 로컬 서버 가동 후 브라우저에서 `http://127.0.0.1:8000/` 접속 확인 (종료는 `Ctrl + C`)

2. settings.py 필수 초기 설정 및 첫 MTV 화면 렌더링 공식

```
# [1] firstpjt/settings.py — 앱 등록 순서 (User App -> Third-Party App -> Django 기본 App 순서 권장)
INSTALLED_APPS = [
    'articles',                 # 내가 만든 로컬 앱을 가장 위에 작성 (끝에 쉼표 ',' 잊지 말 것!)
    'django.contrib.admin',     # 이하 Django 기본 내장 앱들
    ...
]
LANGUAGE_CODE = 'ko-kr'         # 관리자 페이지 및 기본 에러 메시지를 한국어로 변경
TIME_ZONE = 'Asia/Seoul'        # 서버 기준 시간대를 한국 표준시(KST)로 변경
```

```
# [2] firstpjt/urls.py (안내데스크)
from django.urls import path
from articles import views

urlpatterns = [
    path('articles/', views.index),
]
```

```
# [3] articles/views.py (중앙 사령탑)
from django.shortcuts import render

def index(request):  # 제1인자는 무조건 request!
    context = {'name': '홍길동'}
    return render(request, 'articles/index.html', context)

# 주의: 실제 파일 위치는 articles/templates/articles/index.html
```

제 2 단원 : DTL 핵심 문법 · 템플릿 상속 · URL 분배기 요점정리 (교재 2권: Template & URLs)

1. DTL(Django Template Language) 4대 핵심 문법 요약표

구분 및 기호	대표 문법 예시	핵심 작동 규칙 및 주의사항
① 변수 (Variable) `{{ variable }}`	`{{ name }}`, `{{ article.title }}`, `{{ menu.0 }}`	`context` 딕셔너리의 Key 이름으로 접근. 대괄호(`menu[0]`)나 함수 호출 괄호(`article.save()`)는 쓸 수 없으며 **오직 마침표(`.`)로만** 딕셔너리·속성·인덱스에 접근함!
② 필터 (Filter) `{{ var|filter }}`	`{{ content|truncatechars:10 }}` `{{ articles|length }}`, `{{ price|default:0 }}`	변수 값을 화면에 출력하기 전에 가공함. 파이프(`|`) 양옆에 공백 없이 붙여 쓰며, 여러 필터를 연달아 체이닝 가능
③ 태그 (Tag) `{% tag %}`	`{% for a in articles %} ... {% empty %} 글 없음 {% endfor %}` `{% if 조건 %} ... {% endif %}`	반복문·조건문·상속·URL 생성 등 논리 흐름 제어. `for`, `if`, `block`은 반드시 닫는 태그(`endfor`, `endif`, `endblock`)와 짝을 맞춰야 함!
④ 주석 (Comment) `{# ... #}`	`{# 한 줄 주석 #}` `{% comment %} 여러 줄 주석 {% endcomment %}`	HTML 기본 주석(`<!-- -->`)은 브라우저 '페이지 소스 보기'에 그대로 노출되지만, DTL 주석(`{# #}`)은 서버 렌더링 시 완전히 제거되어 보안상 안전함

2. 템플릿 상속(base.html) & 앱별 URL 분리(include) · 역참조(url) 표준 코드

```
# [1] settings.py — 프로젝트 최상위 templates/base.html 경로 등록
TEMPLATES = [{
    ...,
    'DIRS': [BASE_DIR / 'templates'],  # ★ 이 줄을 추가해야 최상위 templates/base.html을 찾을 수 있음!
    'APP_DIRS': True,                  # 각 앱 폴더 내부의 templates/ 폴더 자동 탐색
}]
```

```
# [2] firstpjt/urls.py (루트 URL)
from django.urls import path, include

urlpatterns = [
    path('articles/', include('articles.urls')),
]
```

```
# [3] articles/urls.py (앱 전용 URL)
 from django.urls import path
 from . import views
 app_name = 'articles'  # ★ 성씨(Namespace) 필수!
 urlpatterns = [
     path('', views.index, name='index'),
     path('<int:pk>/', views.detail, name='detail'),
 ]
```

HTML 템플릿에서 주소 부르기(역참조): 하드코딩(`href="/articles/3/"`) 대신 반드시 `<a href="{% url 'articles:detail' article.pk %}">`를 사용하며, View 함수에서 이동할 때도 `return redirect('articles:detail', article.pk)`로 호출합니다.

제 3 단원 : Model 설계 · 마이그레이션 명령어 · Admin 등록 요점정리 (교재 3권: Model)

1. models.py 클래스 정의와 주요 필드 타입 · 옵션 요약

```
# [1] articles/models.py
from django.db import models

class Article(models.Model):
    title = models.CharField(max_length=10)
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

```
# [2] articles/admin.py
from django.contrib import admin
from .models import Article

# 관리자 페이지(/admin/)에 Article 테이블 등록
admin.site.register(Article)
# 최초 생성 시각 고정
# 저장(save)할 때마다 갱신
```

주요 필드 / 옵션	핵심 특징 및 베테랑 개발자의 실전 규칙
`CharField` vs `TextField`	• `CharField(max_length=N)`: 길이 제한이 있는 단문 문자열 (`max_length` **필수 인자!**) • `TextField()`: 글자 수 제한이 없는 장문 텍스트 (게시글 본문 등, `max_length` 선택)
`IntegerField` / `BooleanField`	• `IntegerField()`: 정수 저장 / `BooleanField(default=False)`: `True` 또는 `False` 저장
`auto_now_add` vs `auto_now`	• `auto_now_add=True`: 최초 생성(INSERT) 시점 일시 고정 / `auto_now=True`: 매번 저장(`save`) 시점마다 갱신
`null=True` vs `blank=True`	• `null=True`: DB 컬럼에 `NULL` 허용 (숫자·날짜 필드 전용!) • `blank=True`: 폼 유효성 검사 시 빈칸 허용 (**문자열 필드엔 `null=True` 금지, 오직 `blank=True`만 사용!**)

2. 마이그레이션(Migration) 및 관리자 필수 명령어 5선

명령어	실행 결과 및 실전 활용 시점
`python manage.py makemigrations`	`models.py`의 변경사항을 감지하여 `migrations/0001_initial.py` 설계도 파일 생성 (Git 커밋 대상)
`python manage.py migrate`	설계도 파일들을 실제 DB(`db.sqlite3`)에 테이블로 시공 (최초 실행 시 인증·세션 등 기본 테이블도 함께 생성됨)
`python manage.py showmigrations`	설계도들이 실제 DB에 반영되었는지 확인 (적용 완료된 항목은 `[X]`, 미적용 항목은 `[ ]`로 표시됨)
`python manage.py sqlmigrate articles 0001`	해당 마이그레이션 설계도가 실제로 어떤 SQL문(`CREATE TABLE ...`)으로 번역되는지 화면에 출력

명령어	실행 결과 및 실전 활용 시점
`python manage.py createsuperuser`	`/admin/`에 접속할 관리자 계정 생성 (비밀번호 입력 시 화면에 글자가 안 보여도 정상 입력 중이니 치고 엔터!)

제 4 단원 : UI/UX 3대 법칙 · 웹 폰트 · 반응형 웹(Media Query) 요점정리 (교재 4권: PJT 03)

1. 반응형 웹 구현을 위한 HTML viewport 태그와 CSS @media 표준 공식

```
<!-- [1] HTML <head> 안에 반드시 넣어야 하는 viewport 메타 태그 (스마트폰이 980px 모니터로 착각하는 것 방지) -->
<meta name="viewport" content="width=device-width, initial-scale=1.0">

/* [2] style.css — 웹 폰트(FOIT 방지 swap) & 모바일 퍼스트(min-width 오름차순) 미디어 쿼리 공식 */
@font-face {
    font-family: 'Pretendard';
    src: url('...woff2') format('woff2');
    font-display: swap;  /* 폰트 다운로드 전에는 시스템 기본 폰트를 먼저 보여주고 완료 즉시 교체 */
}
.card-container { display: flex; flex-direction: column; }   /* 기본: 모바일 화면(<768px)은 세로 1열 배치 */
@media (min-width: 768px) {                                  /* 태블릿 이상(>=768px): 2열 배치 */
    .card-container { flex-direction: row; flex-wrap: wrap; }
}
@media (min-width: 1024px) {                                 /* 데스크톱 이상(>=1024px): 최대 폭 고정 */
    .card-container { max-width: 1200px; margin: 0 auto; }
}
```

핵심 개념	시험 및 프로젝트 적용 핵심 요약
UI/UX 3대 심리 법칙	• **힉의 법칙(Hick's Law)**: 선택지가 많으면 결정 시간 증가 → 핵심 메뉴만 노출하고 단계별로 보여주는 점진적 공개 적용 • **피츠의 법칙(Fitts's Law)**: 타깃이 크고 가까울수록 터치가 쉬움 → 모바일 하단 엄지 영역(Thumb Zone)에 최소 48×48px 버튼 배치 • **스켈레톤 UI**: 무한 회전 스피너 대신 콘텐츠 윤곽선 뼈대를 먼저 노출해 체감 대기 시간 단축
GitHub Pages 배포 주의점	• 저장소 최상위(Root)에 반드시 소문자 `index.html`이 있어야 함 (Windows와 달리 Linux 서버는 `Image.png`와 `image.png` 대소문자를 엄격히 구분함!) • HTML/CSS/JS만 제공하는 **정적(Static) 호스팅**이므로 Python과 DB가 필요한 Django 서버는 작동 불가

제 5 단원 : ORM과 QuerySet API · Field Lookups 요점정리 (교재 5권: ORM)

1. QuerySet API를 활용한 DB 조작(CRUD) 핵심 코드 공식

작업 구분	파이썬 ORM 표준 코드	반환값 타입 및 베테랑 핵심 체크포인트
Create (생성)	[권장] `article = Article(title='제목', content='내용')` `article.save()` [단축] `Article.objects.create(title='제목', content='내용')`	`save()` 호출 전에는 `article.pk`가 `None`이다가, `save()`가 실행되어 DB에 `INSERT`되는 순간 `pk`(1, 2...)가 부여됨!

Read (전체/조건)	`Article.objects.all()`
	`Article.objects.filter(title__contains='장고')`
	`Article.objects.exclude(pk=1).order_by('-pk')`
		무조건 `QuerySet`(리스트형 바구니) 반환! 결과가 0개면 빈 바구니 `[]`를 반환하며 에러가 나지 않음.
		`order_by('-pk')`는 최신순(내림차순) 정렬

Read (단일)	`article = Article.objects.get(pk=1)`	**바구니가 아닌 단일 모델 인스턴스(알맹이) 반환!** 결과가 0개면 `DoesNotExist`, 2개 이상이면 `MultipleObjectsReturned` 에러 발생 → **오직 고유값(`pk`) 조회에만 사용!**

작업 구분	파이썬 ORM 표준 코드	반환값 타입 및 베테랑 핵심 체크포인트

Update (수정)	`article = Article.objects.get(pk=1)`
	`article.title = '수정된 제목'`
	`article.save()`
		이미 `article.pk == 1`이 존재하는 객체이므로, 속성 변경 후 `save()`를 호출하면 새 행 추가가 아니라 기존 1번 행 덮어쓰기(`UPDATE`)가 실행됨

Delete (삭제)	`article = Article.objects.get(pk=1)` `article.delete()`	DB에서 해당 행을 즉시 삭제(`DELETE`)하며, 삭제된 후에는 그 번호(`pk=1`)는 재사용되지 않고 영구 결번됨

2. 자주 쓰는 Field Lookups (SQL WHERE 조건절 매핑)

Field Lookup 문법	변환되는 SQL 의미	설명 및 활용 예시
`title__exact='hello'`	`WHERE title = 'hello'`	정확히 일치 (`filter(title='hello')`와 동일, 대소문자 구분 시 사용)
`title__contains='dja'`	`WHERE title LIKE '%dja%'`	특정 문자열 포함 여부 검색 (대소문자 무시할 땐 `__icontains`)
`title__startswith='첫'`	`WHERE title LIKE '첫%'`	특정 글자로 시작하는 데이터 검색 (끝나는 글자는 `__endswith`)
`pk__gt=3` / `pk__gte=3`	`WHERE id > 3` / `WHERE id >= 3`	Greater than(초과) / Greater than or equal(이상) (미만: `__lt`, 이하: `__lte`)

제 6 · 7 단원 : Django Form · ModelForm과 통합 CRUD View 표준 코드 (교재 6·7권: ORM with View & Form)

1. articles/forms.py 정의와 조회·삭제(index · detail · delete) 표준 코드

교재 6권의 수동 7대 View(`index`, `detail`, `new`, `create`, `edit`, `update`, `delete`)는 교재 7권에서 `ModelForm` 도입과 함께 5개 함수(`index`, `detail`, `create`, `update`, `delete`)로 깔끔하게 통합됩니다.

```
# [1] articles/forms.py — ModelForm 및 Widget 정의
from django import forms
from .models import Article

class ArticleForm(forms.ModelForm):
    class Meta:
        model = Article
        fields = '__all__'  # 전체 필드 사용 (특정 필드 제외 시: exclude = ('title',))
```

```
# [2] articles/views.py — 목록 조회(index), 상세 조회(detail), 글 삭제(delete)
from django.shortcuts import render, redirect
from .models import Article
from .forms import ArticleForm

def index(request):                                    # 1. 전체 목록 조회 (GET)
    articles = Article.objects.all()
    return render(request, 'articles/index.html', {'articles': articles})

def detail(request, pk):                               # 2. 단일 상세 조회 (GET)
    article = Article.objects.get(pk=pk)
    return render(request, 'articles/detail.html', {'article': article})
```

```
def delete(request, pk):                               # 3. 글 삭제 (반드시 POST 요청 + csrf_token으로만 처리!)
    article = Article.objects.get(pk=pk)
    article.delete()
    return redirect('articles:index')
```

2. 통합 생성(create) & 통합 수정(update) View 완성형 템플릿

```
def create(request):                                    # 4. 새 글 작성 (new GET + create POST 통합!)
    if request.method == 'POST':
        form = ArticleForm(request.POST)                # POST: 사용자 입력 데이터 채움
        if form.is_valid():                             # 서버 측 유효성 검문 통과 시
            article = form.save()                       # 새 게시글 DB INSERT 후 객체 반환
            return redirect('articles:detail', article.pk)  # PRG 패턴: 상세 페이지로 302 리다이렉트!
    else:
        form = ArticleForm()                            # GET: 빈 입력창 폼 생성
    context = {'form': form}                            # ★ GET 진입 시 & POST 유효성 실패 시 공동 합류점!
    return render(request, 'articles/create.html', context)
```

```
def update(request, pk):                                # 5. 기존 글 수정 (edit GET + update POST 통합!)
    article = Article.objects.get(pk=pk)                # 수정할 기존 게시글 먼저 조회
    if request.method == 'POST':
        form = ArticleForm(request.POST, instance=article)  # ★ instance=article 키워드 인자 필수!
        if form.is_valid():
            form.save()                                 # 기존 게시글 DB UPDATE
            return redirect('articles:detail', article.pk)
    else:
        form = ArticleForm(instance=article)            # ★ GET: 기존 글 내용이 채워진 수정 폼 생성!
    context = {'article': article, 'form': form}
    return render(request, 'articles/update.html', context)
```

베테랑 개발자의 실전 체크리스트 — 템플릿(HTML) 작성 시 절대 빼먹으면 안 되는 2가지!
① **`<form action="..." method="POST">`** 바로 아랫줄에 **`{% csrf_token %}`** 필수! (누락 시 **`403 Forbidden`** 에러 발생)
② **`update.html`**의 **`action`** 주소에는 반드시 수정할 글의 번호(**`pk`**)를 함께 넘겨야 함! 예: **`<form action="{% url 'articles:update' article.pk %}" method="POST">`** (**`create.html`**은 아직 **`pk`**가 없으므로 **`{% url 'articles:create' %}`**만 작성)

제 8 · 9 단원 : Custom User 설정과 인증(Authentication) 6대 기능 코드 바이블 (교재 8·9·10권)

1. 첫 migrate 전 Custom User 3단계 세팅 & 커스텀 ModelForm 정의

```
# [단계 1] accounts/models.py                       # [단계 2] firstpjt/settings.py
from django.contrib.auth.models import AbstractUser  AUTH_USER_MODEL = 'accounts.User'

class User(AbstractUser):                            # [단계 3] accounts/admin.py
    pass                                             from django.contrib import admin
                                                     from django.contrib.auth.admin import UserAdmin
                                                     from .models import User
                                                     admin.site.register(User, UserAdmin)
```

```
# [단계 4] accounts/forms.py — get_user_model()로 커스텀 User 모델 연결 및 일반 회원 수정 필드 제한
from django.contrib.auth import get_user_model
from django.contrib.auth.forms import UserCreationForm, UserChangeForm

class CustomUserCreationForm(UserCreationForm):
    class Meta(UserCreationForm.Meta):
        model = get_user_model()  # 현재 활성화된 AUTH_USER_MODEL('accounts.User')을 자동 반환

class CustomUserChangeForm(UserChangeForm):
    class Meta(UserChangeForm.Meta):
        model = get_user_model()
        fields = ('first_name', 'last_name', 'email')  # ★ UserChangeForm은 관리자용이라 모든 권한 필드가 노출되므로 제한 필수!
```

2. 인증 6대 기능 PART A: 로그인(login) · 로그아웃(logout) · 회원가입(signup)

```
# accounts/views.py (상단 import 및 로그인·로그아웃·회원가입)
from django.shortcuts import render, redirect
from django.contrib.auth import login as auth_login, logout as auth_logout, update_session_auth_hash
from django.contrib.auth.forms import AuthenticationForm, PasswordChangeForm
from django.contrib.auth.decorators import login_required
from .forms import CustomUserCreationForm, CustomUserChangeForm

def login(request):                                    # 1. 로그인 (AuthenticationForm -> 제1인자 request!)
    if request.user.is_authenticated:
        return redirect('articles:index')
    if request.method == 'POST':
        form = AuthenticationForm(request, request.POST)
        if form.is_valid():
            auth_login(request, form.get_user())       # 세션 생성 및 브라우저에 sessionid 쿠키 발급!
            return redirect(request.GET.get('next') or 'articles:index')
    else:
        form = AuthenticationForm()
    return render(request, 'accounts/login.html', {'form': form})

@login_required
def logout(request):                                   # 2. 로그아웃 (세션 금고 및 쿠키 열쇠 파기)
    auth_logout(request)
    return redirect('articles:index')
```

```
def signup(request):                                   # 3. 회원가입 (CustomUserCreationForm -> 제1인자 req
uest.POST!)
    if request.user.is_authenticated:
        return redirect('articles:index')
    if request.method == 'POST':
        form = CustomUserCreationForm(request.POST)
        if form.is_valid():
            user = form.save()                         # 새 회원 DB 저장 후
            auth_login(request, user)                  # 가입과 동시에 자동 로그인까지 처리!
            return redirect('articles:index')
    else:
        form = CustomUserCreationForm()
    return render(request, 'accounts/signup.html', {'form': form})
```

3. 인증 6대 기능 PART B: 회원탈퇴(delete) · 정보수정(update) · 비번변경(password)

```
@login_required
def delete(request):                                   # 4. 회원 탈퇴 (★ 반드시 delete 먼저, logout 나중!)
    request.user.delete()                              # ① 누구인지 알 때 DB에서 회원 정보 먼저 삭제
    auth_logout(request)                               # ② 남아있는 세션 금고 정리
    return redirect('articles:index')

@login_required
def update(request):                                   # 5. 회원정보 수정 (instance=request.user 필수!)
    if request.method == 'POST':
        form = CustomUserChangeForm(request.POST, instance=request.user)
        if form.is_valid():
            form.save()
            return redirect('articles:index')
    else:
        form = CustomUserChangeForm(instance=request.user)
    return render(request, 'accounts/update.html', {'form': form})
```

```
@login_required
def password(request):                    # 6. 비밀번호 변경 (기본 안내 경로: /accounts/password/)
    if request.method == 'POST':
        form = PasswordChangeForm(request.user, request.POST)  # ★ 제1인자 request.user 필수!
        if form.is_valid():
            user = form.save()
            update_session_auth_hash(request, user)  # ★ 비번 변경 후 세션 유지(로그아웃 방지)!
            return redirect('articles:index')
    else:
        form = PasswordChangeForm(request.user)
    return render(request, 'accounts/password.html', {'form': form})
```

제 10 단원 : 비밀번호 암호화(PBKDF2) · 비밀번호 초기화 · 4대 Form 인자 총정리 (교재 10권)

1. 부모 클래스에 따른 4대 Form 생성자 인자 순서 완벽 비교표 (교재 10권 핵심)

Form 클래스명	부모 클래스	첫 번째 위치 인자	POST 요청 시 표준 호출 코드 및 핵심 주의사항
`ArticleForm` `CustomUserCreationForm`	`ModelForm`	`data`	`ArticleForm(request.POST)` 새 행을 DB에 `INSERT`하므로 제1인자로 `request.POST` 전달
`CustomUserChangeForm`	`ModelForm`	`data` (+ `instance`)	`CustomUserChangeForm(request.POST, instance=request.user)` `instance`는 9번째 인자이므로 반드시 키워드 인자로 지정!
`AuthenticationForm`	`Form`	`request`	`AuthenticationForm(request, request.POST)` 세션 인증 처리를 위해 제1인자로 `request`, 제2인자로 `data` 전달
`PasswordChangeForm`	`SetPasswordForm` (일반 `Form`)	`user`	`PasswordChangeForm(request.user, request.POST)` 부모인 `SetPasswordForm`의 생성자 정의에 따라 제1인자가 `user`!

2. 비밀번호 암호화(단방향 해시 · 솔트 · 키 스트레칭 PBKDF2) 핵심 공식 요약

암호화 개념 / 공격·방어	핵심 원리 및 시험·실무 필수 체크포인트
인코딩 vs 암호화 vs 해시	• **인코딩(Base64)**: 비밀키 없이 누구나 즉시 원본 복원 가능 (보안성 0%, 평문 저장 시 **Credential Stuffing** 2차 피해 유발) • **암호화(Encryption)**: 비밀키로 복호화 가능한 양방향 변환 / • **해시(Hash)**: 복원이 불가능한 **단방향 변환** (비밀번호 저장 표준!)
해시 함수 3대 특성 & SHA-256	① **고정된 길이**(원래 비밀번호 길이 은폐 및 DB 저장 공간 일정 유지) / ② **단방향성**(원본 역추적 불가) / ③ **눈사태 효과(Avalanche Effect)**(점 하나만 달라도 해시값 완전 변경). 단, `SHA-256`(64글자) 단독 사용은 **연산 속도가 너무 빨라** 공격에 취약함!
레인보우 테이블 ↔ 솔트(Salt)	• **레인보우 테이블 공격**: 자주 쓰는 비밀번호 수억 개의 해시값을 미리 계산해 둔 정답지 사전으로 대조하는 공격 • **방어책 — 솔트(Salt)**: 사용자마다 고유한 임의 문자열(솔트)을 비밀번호에 붙여 해싱! (같은 비밀번호도 유저마다 해시값이 달라지며, 해커가 DB에서 솔트를 알아내도 그 유저 1명만을 위한 정답지를 새로 만들어야 하므로 공격 무력화)
무차별 대입 ↔ 키 스트레칭	• **무차별 대입 공격(Brute-force)**: 고성능 GPU의 빠른 연산 속도로 가능한 모든 비밀번호 조합을 하나씩 직접 대입하는 공격 • **방어책 — 키 스트레칭(Key Stretching)**: 해시 연산을 수십만 번 반복(`PBKDF2`, `Argon2`, `bcrypt`)해 1회 검증을 약 0.2초로 지연 (정상 유저는 체감 못 하지만, 10억 개 대입 시 1회 연산 17분 → 100만 회 반복 시 **3.2년 이상**으로 지연!)

암호화 개념 / 공격·방어	핵심 원리 및 시험·실무 필수 체크포인트
Django 비밀번호 저장 포맷	• 포맷: `<algorithm>$<iterations>$<salt>$<hash>` (예: `pbkdf2_sha256$600000$U7e5k...$cBWsU...=`) • **[시험 1순위 함정]** 비밀번호 해시 문자열 안에 **사용자 아이디(`username`)는 절대 포함되지 않는다!**

3. 내장 비밀번호 초기화(password_reset) 설정 코드와 URL 배치 순서 규칙

```
# [1] crud/urls.py — 내장 인증 URL(django.contrib.auth.urls) 포함 및 우선순위 배치 규칙
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('articles/', include('articles.urls')),
    path('accounts/', include('accounts.urls')),             # ★ 1순위: 우리 accounts.urls를 윗줄에 배치!
    path('accounts/', include('django.contrib.auth.urls')),  # ★ 2순위: 내장 auth.urls를 아랫줄에 배치!
]
# [2] crud/settings.py — 실제 메일 서버 없이 터미널(콘솔)에서 비밀번호 재설정 이메일 링크를 확인하는 설정
EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'
```

- **왜 같은 `'accounts/'` prefix로 두 번 `include`할 때 우리 앱을 위에 적을까?**: Django는 위에서부터 순서대로 URL을 탐색하므로, `login/`, `logout/`처럼 겹치는 경로에서 우리가 작성한 View가 우선 실행되게 하려면 `accounts.urls`를 먼저 적어야 합니다.
- **비밀번호 초기화 4단계 내장 경로**: ① 이메일 입력(`/accounts/password_reset/`) → ② 발송 안내(`/accounts/password_reset/done/`) → ③ 콘솔 링크 접속 및 새 비번 입력(`/accounts/reset/<uidb64>/<token>/`) → ④ 완료(`/accounts/reset/done/`). 이때 **가입되지 않은 이메일을 입력해도 계정 존재 여부 악용(User Enumeration)을 막기 위해 에러 없이 동일한 완료 화면**을 보여줍니다.

4. 베테랑 개발자의 실전 족보: User 모델을 참조하는 2가지 구분 공식 (models.py vs 그 외 파일)

작성 파일 위치	사용해야 하는 함수 / 설정값	왜 이렇게 구분해서 써야 할까? (실행 순서 원리)
`models.py` 내부에서만! (외래키 `ForeignKey` 등 정의 시)	`settings.AUTH_USER_MODEL` (문자열 `'accounts.User'`)	앱이 구동되는 중에 `models.py`가 아직 다 읽히지 않았을 수 있으므로, 클래스 객체 대신 문자열 이름표로 먼저 연결해 두어야 순환 참조 에러가 나지 않음!
그 외 모든 파이썬 파일! (`forms.py`, `views.py` 등)	`get_user_model()` (실제 `User` 클래스 객체 반환)	`models.py` 로딩이 끝난 뒤 실행되는 파일들이므로, 현재 활성화된 진짜 `User` 모델 클래스를 동적으로 가져오는 `get_user_model()` 함수를 사용함

제 11 단원 : PJT-04 Model · Form · Auth 실전 심화 코드 바이블 (교재 11권)

1. Model Fields 심화: choices 선택지 · get_FOO_display() · PositiveIntegerField 이중 방어

```
# todos/models.py — choices 튜플 리스트 & PositiveIntegerField(0 이상 정수 DB CHECK 제약)
class Todo(models.Model):
    STATUS_CHOICES = [('TODO', '할 일'), ('DOING', '진행 중'), ('DONE', '완료')]
    PRIORITY_CHOICES = [(1, '낮음'), (2, '보통'), (3, '높음')]
    TAG_CHOICES = [('WORK', '업무'), ('STUDY', '공부'), ('LIFE', '생활')]

    title = models.CharField(max_length=100, verbose_name='제목')
    content = models.TextField(blank=True, verbose_name='내용')  # 문자열엔 blank=True만 사용!
    status = models.CharField(max_length=5, choices=STATUS_CHOICES, default='TODO')
    priority = models.PositiveIntegerField(choices=PRIORITY_CHOICES, default=2)
    tags = models.CharField(max_length=100, blank=True)          # 'WORK,STUDY' 콤마 문자열 저장
```

```
    def get_tag_list(self):          # 'WORK,STUDY' -> ['WORK', 'STUDY'] 리스트로 분리
        return self.tags.split(',') if self.tags else []

    def get_tags_display(self):      # 'WORK,STUDY' -> '업무, 공부' 한글 레이블로 변환
        labels = dict(self.TAG_CHOICES)
        return ', '.join(labels.get(tag, tag) for tag in self.get_tag_list())
```

템플릿에서 `choices` 레이블 출력 공식: `{{ todo.priority }}`는 DB 값(`3`)을 출력하므로, 화면용 이름(`'높음'`)을 출력할 때는 자동 생성 메서드인 `{{ todo.get_priority_display }}`, `{{ todo.get_status_display }}`(괄호 없음!)를 사용합니다.

2. Form 심화: GET 검색 폼(cleaned_data) · Form 상속 · 다중 선택(MultipleChoiceField)

```
# [1] todos/forms.py — 검색용 일반 Form, 다중 체크박스(MultipleChoiceField), 수정 폼 상속
class TodoSearchForm(forms.Form):    # DB에 저장하지 않는 검색 조건용 폼 -> 일반 forms.Form 사용!
    status = forms.ChoiceField(choices=[('', '전체')] + Todo.STATUS_CHOICES, required=False)
    keyword = forms.CharField(required=False, widget=forms.TextInput(attrs={'class': 'form-control'}))
```

```
class TodoForm(forms.ModelForm):
    tags = forms.MultipleChoiceField(choices=Todo.TAG_CHOICES, widget=forms.CheckboxSelectMultiple)

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        if self.instance.pk:         # ★ 수정 창 진입 시: 'WORK,STUDY' -> 리스트로 체크 상태 복원!
            self.initial['tags'] = self.instance.get_tag_list()

    class Meta:
        model = Todo
        fields = ['title', 'content', 'status', 'priority', 'tags']
        widgets = {'title': forms.TextInput(attrs={'class': 'form-control'})}  # 모델 필드는 인테리어!
```

```
class UpdateTodoForm(TodoForm):      # ★ Form 상속: TodoForm을 물려받고 'completed'만 추가!
    class Meta(TodoForm.Meta):
        fields = TodoForm.Meta.fields + ['completed']
```

```
# [2] todos/views.py — 검색(cleaned_data), 다중 태그 콤마 합치기(','.join), 일회용 알림(messages.success)
from django.contrib import messages

def index(request):                  # GET 검색 처리 (method="GET"이므로 csrf_token 불필요)
    todos = Todo.objects.all()
    form = TodoSearchForm(request.GET)
    if form.is_valid():              # 보안 검색대 통과 후 cleaned_data 딕셔너리에서 안전하게 값 추출!
        if form.cleaned_data['status']:
            todos = todos.filter(status=form.cleaned_data['status'])
        if form.cleaned_data['keyword']:
            todos = todos.filter(title__contains=form.cleaned_data['keyword'])
    return render(request, 'todos/index.html', {'todos': todos, 'form': form})
```

```
def create(request):                 # 리스트 ['WORK', 'STUDY'] -> 'WORK,STUDY' 변환 저장
    if request.method == 'POST':
        form = TodoForm(request.POST)
        if form.is_valid():
            todo = form.save(commit=False)                          # DB 저장을 잠시 멈추고 객체만 받은 뒤
            todo.tags = ','.join(form.cleaned_data['tags'])         # 리스트를 콤마 문자열로 변환!
            todo.save()
            messages.success(request, '할 일을 추가했습니다.')   # 1회용 포스트잇 알림 부착!
            return redirect('todos:index')
```

3. Widget 지정 2가지 방식 비교 & Auth 심화(안전한 ?next= 복귀 · password=None · @require_POST)

구분	방법 A: `Meta.widgets = {'필드': ...}` (인테리어만 바꾸기)	방법 B: 클래스 상단에 필드 재선언 (가구 새로 사기)
사용 대상 및 범위	**`ModelForm` 전용** — 모델(`models.py`)에 이미 있는 필드(`title`, `username` 등)의 HTML 모양만 바꿀 때 사용	**`Form` · `ModelForm` 모두** — 모델에 없는 칸(검색 `keyword`, 다중 `tags`, 회원가입 `password1`·`password2`)에 사용
기존 설정 유지 여부	모델 필드의 `max_length`, `label`, `help_text`가 **그대로 유지됨**	기존 설정이 **모두 초기화되어 사라지므로** `label='비밀번호'` 등을 반드시 다시 적어주어야 함!

```
# [1] accounts/forms.py — CustomUserChangeForm에서 지저분한 영어 비밀번호 안내 칸 제거 (password = None)
class CustomUserChangeForm(UserChangeForm):
    password = None                  # ★ 수정 폼에서 비밀번호 해시 안내 문구 칸을 깔끔하게 제거!
    class Meta(UserChangeForm.Meta):
        model = get_user_model()
        fields = ['email', 'first_name', 'last_name']
```

```python
# [2] accounts/views.py — 오픈 리다이렉트 방어(url_has_allowed_host_and_scheme) & @require_POST 보호
from django.utils.http import url_has_allowed_host_and_scheme
from django.views.decorators.http import require_POST

def login(request):                    # 로그인 성공 후 안전한 ?next= 주소 검증 및 복귀
    if request.method == 'POST':
        form = CustomAuthenticationForm(request, request.POST)
        if form.is_valid():
            auth_login(request, form.get_user())
            next_url = request.GET.get('next')
            if next_url and url_has_allowed_host_and_scheme(next_url, {request.get_host()}):
                return redirect(next_url)  # ★ 우리 사이트 내부 주소일 때만 안전하게 이동!
            return redirect('todos:index')
# 템플릿 login.html의 action에는 ?next={{ request.GET.next|urlencode }} 필터로 & 기호 보호!
```

```python
@login_required
@require_POST                          # ★ GET 요청(주소창/이미지 태그) 원천 차단 (405 반환)!
def delete(request):
    request.user.delete()
    auth_logout(request)
    return redirect('todos:index')
```

4. [교재 11권 부록 코드] TextChoices 클래스 · base.html 메시지 알림창 · 프로필(user_profile) 표준 코드

```python
# [1] todos/models.py — Django 3.0+ 현업 표준 TextChoices / IntegerChoices 열거형 클래스 방식
class Todo(models.Model):
    class Status(models.TextChoices):
        TODO = 'TODO', '할 일'
        DOING = 'DOING', '진행 중'
        DONE = 'DONE', '완료'
    status = models.CharField(max_length=5, choices=Status.choices, default=Status.TODO)
```

```python
# [2] templates/base.html — Django Message Framework 일회용 포스트잇 알림 출력 템플릿 코드
{% if messages %}
  {% for message in messages %}
    <div class="alert alert-{{ message.tags }}">{{ message }}</div>
  {% endfor %}
{% endif %}
# 팁: error 등급과 alert-danger 색상을 맞추려면 settings.py에 MESSAGE_TAGS 설정!
```

```python
# [3] accounts/views.py — 프로필 페이지 조회 시 기본 {{ user }}(로그인한 사용자)와 이름 충돌 방지 공식
@login_required
def profile(request, username):
    User = get_user_model()
    person = User.objects.get(username=username)
    todos = Todo.objects.all()
    context = {'user_profile': person, 'todos': todos}  # ★ 'user' 키 충돌 방지!
    return render(request, 'accounts/profile.html', context)
```

5. 최종 실전 점검 — 프로젝트 시작부터 인증·보안까지 막히지 않는 베테랑의 7계명

- **1계명**: 새 앱을 만들면 코딩하기 전에 무조건 **`settings.py`**의 **`INSTALLED_APPS`**부터 열고 맨 윗줄에 등록한다.
- **2계명**: 첫 번째 **`migrate`**를 실행하기 전에 무조건 **`accounts`** 앱의 **`class User(AbstractUser): pass`**와 **`AUTH_USER_MODEL = 'accounts.User'`** 부터 설정한다.
- **3계명**: 데이터를 바꾸는 요청(생성·수정·삭제·로그아웃·탈퇴·비번변경)은 무조건 **`POST`** + **`{% csrf_token %}`** + **`redirect()`** 3종 세트로 묶는다.
- **4계명**: 4대 폼의 제1인자(**`ModelForm`**은 **`request.POST`**, **`AuthenticationForm`**은 **`request`**, **`PasswordChangeForm`**은 **`request.user`**)를 절대 혼동하지 않는다.
- **5계명**: 회원정보 수정(**`CustomUserChangeForm`**) 시 **`fields`**를 일반 필드로 제한하고, 비밀번호 변경 직후엔 반드시 **`update_session_auth_hash(request, user)`**를 호출한다.
- **6계명**: 비밀번호 초기화(**`django.contrib.auth.urls`**)를 루트 **`urls.py`**에 추가할 때는 반드시 우리가 만든 **`include('accounts.urls')`** 아랫줄에 배치한다!
- **7계명**: 모델에 있는 필드를 꾸밀 땐 **`Meta.widgets`**를 쓰고, 모델에 없는 칸(**`keyword`**, **`tags`**, **`password1·2`**)을 꾸밀 땐 필드를 재선언(**`label`** 포함!)하며, **`?next=`** 이동 시엔 반드시 **`url_has_allowed_host_and_scheme`**으로 검문한다!